הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על **שש שאלות** רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

מה התיאור המתאים ביותר למהלך טיפול בפסיקת החומרה? מגיע ל- interrupt handler ע"י dispatching

[✓ א.] פסיקה מתקבלת ישירות ע״י גרעין מערכת ההפעלה ומטופלת על ידו.

ב. פסיקה מתקבלת ע״י המעבד שמתחיל לבצע קטע קוד טיפול בפסיקה בתוך הגרעין. ×

ג. פסיקה מתקבלת ע״י המעבד שמתחיל לטפל בה במרחב משתמש של תהליך יעד הפסיקה. ×

ד. פסיקה מתקבלת ע״י תוכנית שירות של מערכת הפעלה שבהתאם למספר הפסיקה מחליטה האם לטפל בה בעצמה או להעביר את הטיפול לגרעין.

בכל מקרה יש להעביר לגרעין

שאלה 2

איזו תכונה נדרשת למעבד CPU בשביל שיהיה ניתן לממש עליו את ביצועי וירטואליזציה בשיטת Para-virtualization?

א. המעבד פועל אך ורק ב kernel mode.

ב. כל הפקודות המיוחסות (privileged instructions , הגורמות ל TRAP) מהוות תת-קבוצה של הפקודות הרגישות(sensitive instructions) במערכת.

ג. כל הפקודות הרגישות (sensitive instructions) במערכת מהוות תת-קבוצה של פקודות מיוחסות (privileged instructions , הגורמות ל TRAP).

[✓ ד.] אף תשובה איננה נכונה.

שאלה 3

מהו ההבדל בין קישור סימבולי (SOFT LINK) וקישור קשיח (HARD LINK)?

[✓ א.] ל-SOFT LINK ולקובץ שהוא מקשר יש I-NODE -ים שונים ול HARD LINK ולקובץ אותו I-NODE

✗ ב. פתיחת הקובץ דרך SOFT LINK מהירה יותר מאשר דרך HARD LINK

✗ ג. יכול להיות ש HARD LINK מצביע על הקובץ שכבר נמחק מה שלא יכול להיות עם SOFT LINK והפך

✗ ד. יצירת HARD LINK דורשת יצירת קובץ נוסף שיכיל את הקישור וב SOFT LINK לא דורשת.

שאלה 4

איזו אחת מהבעיות הבאות קיימת כאשר משתמשים בתהליכוני משתמש - user threads ?

[✓ א.] חסימת תהליכון אחד תגרום לחסימת כל התהליך. כן

✗ ב. החלפה בין תהליכונים (context switch) דורשת מעבר לגרעין המערכת.

ג. לא קיימת אפשרות יצירת משתנים לוקאליים לתהליכון.

ד. לא קיימת אפשרות לשלוט על כמות הזמן שכל תהליכון מקבל.

שאלה 5

אלגוריתם הבנקאי הוא:

✗ א. אלגוריתם המונע deadlock על ידי שבירת האפשרות למעגל בגרף התלויות.

✗ ב. אלגוריתם המונע deadlock על ידי הכתבת סדר חד משמעי לביצוע בקשות של תהליכים שונים.

[✓ ג.] אלגוריתם המתחמק מ deadlock על ידי בדיקה שבכל שלב קיים סדר כלשהו המאפשר הרצה וסיום כל התהליכים הקיימים במערכת.

ד. אלגוריתם המאפשר להיחלץ מצב deadlock עם פגיעה מינימלית בתהליכים.

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

- גודל הבלוק במערכת הקבצים הוא 4 Kbytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

[illegible]
[illegible] 1B

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב 3076 בלוקים בסה״כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) .

הערה: לצורך החישוב 1 KByte = 1024 Byte , 1 MByte = 1024 KByte

א. 12292 Kbytes

[✓ ב.] 12288 Kbytes

ג. 12328Kbytes

ד. 12332Kbytes

המשך הבחינה בעמוד הבא

4kB

4B

40kB

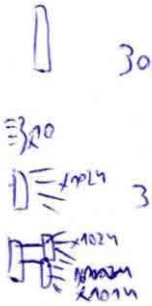
~~[illegible]~~

10
1024
2038 } × 4kB

3066

1024

[illegible]



3065

2047

~~[illegible]~~

3072 × 4kB = 12,288 kB

[illegible]
2040 [illegible]
2038

נא לא לכתוב בשוליים

1.30

45
20
20

חלק ב (55 נקודות)

ענו על השאלות 7-9.

שאלה 7 (25 נקודות)

א. (7 נקודות)

נתונה התוכנית להלן עבור בעיית יצרן-צרכן(יצרן יחיד וצרכן יחיד) שלא משתמשת באמצעי סנכרון של מערכת.

הגבול העליון של המאגר הוא N והמאגר ריק בתחילה.

```
typedef struct {. . .} item;
item buffer[N];
int in = 0;
int out = 0;

ProducerFunction() {
item produced;
while (true) {
        /* produce an item in "produced" */
        while (((in + 1) % N) == out) ;  /* do nothing, busy wait */
        buffer[in] = produced;
        in = (in + 1) % N;
    }
}

ConsumerFunction() {
item consumed;
while (true) {
        while (in == out)  ; /* do nothing, busy wait */
        consumed = buffer[out];
        out = (out + 1) % N;
        /* consume the item in "consumed" */
    }
}
```

כל עוד out אחד לפני

כל עוד in [illegible]

אם צרכן אין לו צרך מוצר, יצרן יכול לזוז
צריך (ימצא) את המוצר הבא (לא היה N מוצרים לא נצרכו)
ולכן אין צורך להמתין

אם יצרן אין לו יצר מוצר (buffer) הצרכן
יחכה לו (זה מובן שצרכן את כל המוצרים ויחכה לחדש
מוצרים חדשים)

מוצרים בגודל מערך:
0,...,N-1

לא ימתינו
לנצח ולא יהיה קיפאון
כי לא יכנס לבעיה
outב מצב לכל זמן ולא [illegible]

(נכון)
אם יש הרבה מוצרים
היצרן יכניס ויצר

(המשך השאלה בעמוד הבא)

- קטע יצרן-צרכן
- מניעה הדדית
- לא יהיה קפאון (הרעבה)
- אין קיפאון (הרעבה) במצב מוכן
- מחוץ לקטע קריטי לא יעצור אחרים
והכנס

מה שצריך
שנכנס
לכמה מוצרים יש במצב מאגר? אחד מוצר

כשיש N
זה יכול מבחין לייצר?
רק צרכן

שאלון 514

87.72.9 M2

[illegible]

הסבירו איך התוכנית פותרת(כולל כל תנאי פתרון קטע קריטי) את בעיית יצרן-צרכן או הפריכו ע"י דוגמא נגדית.

היצרן והצרכן למאגר עוברים באופן מעגלי ($0,\dots,N-1,0,\dots$) כאשר in מסמן שינוקם ליצור ~~[illegible]~~ היצרן, ו-out לצרכן.

נראה שהתוכנית פותרת את בעיית יצרן-צרכן וכן מניעה הדדית - אם יצרן עוד לא יצר מוצר (in==out) הצרכן יחכה לו while

כלומר לא ייתכן שהמאגר ריק (המצב [illegible] לו הצרכן [illegible] כמו היצרן) והצרכן ימשיך לצרוך מוצרים

כמו כן, אם הצרכן עוד לא צרך N-1 מוצרים, היצרן יחכה לו ולא ימשיך לייצר (כלומר יגרום ויצרים את

המוצר הישן שעוד לא נצרך) (in+1)%N==out [illegible] יצרן [illegible], כלומר out אחד לפני in בסדר המעגלי.

בכך לעולם לא יקרה in==out, ולכן יש מניעה הדדית. אין קיפאון והרעבה - לא ייתכן ששניהם מחכים

בהמתנה עסוקה כי לא ייתכן ש-out אחד לפני in וגם in==out (התנאים להמתנה), ולכן בהכרח אחד מהם יכול להתקדם; לכן גם יש המתנה חסומה — (אין הרעבה של אחד התהליכים). בנוסף כמובן שתהליך מחוץ לקטע הקריטי

(מצב [illegible] לצרכן [illegible]) לא ימנע מהאחר מלהיכנס ולהתקדם, [illegible] בגלל [illegible] המתנה, ובנוסף [illegible]

[illegible] המעבד [illegible] כמובן. לכן המצב [illegible] buffer [illegible] בין 0 ל-N-1 מוצרים (ולא N כפי [illegible])

ב. (4 נקודות)

מה החיסרון המובנה בפתרון שבסעיף א' מעבר להמתנה פעילה?

תשובה והסבר:

מעבר כמובן להמתנה הפעילה, קיים חסרון מובנה שבו, כפי שהסברתי, במאגר

יכולים להיות רק N-1 מוצרים ולא N (הוצג כדי [illegible] עד [illegible]), ולכן אם

~~[illegible]~~ היצרן יכול לייצר עוד מוצר אחר, הוא מנוע מלעשות זאת, גם אם היינו רוצים שבחינתנו

תוכן הוא ייצור N ברצף. ~~[illegible]~~ חיסרון מובהק [illegible] הוא יעילות במקום, בגוון

כלומר גם אם במאגר יש גודל מסוים סה"כ N מוצרים, עדיין אנו שומרים מקום (מערך) עבור N מוצרים, ויתרה מזאת גם (ברשימה) [illegible] כולם [illegible] היצרן/צרכן פועלים [illegible].

כלומר "ההדמיה" של מאגר בגין מערך אינה נחוצה.

ג. (7 נקודות)

נתונה התוכנית להלן עבור בעיית יצרנים-צרכנים(כמה יצרנים וכמה צרכנים) שמשתמשת באמצעי סנכרון של מערכת(סמפורים בינאריים).

- אין גבול העליון למאגר והוא ריק בתחילה.
- המאגר מנוהל כרשימה מקושרת כאשר פונקציה Enqueue(item) מכניסה לסוף הרשימה ו Dequeue() מוציאה מראש הרשימה. שתי הפונקציות מעדכנות את המצביעים לראש וסוף הרשימה בהתאם. שתי הפונקציות **לא משתמשות באמצעי הסנכרון**.

נא לא לכתוב בשוליים

משתנים משותפים:

```
typedef struct {. . .} item;
int counter = 0 ; // items counter
binary_semaphore sem = 1; // binary semaphore
binary_semaphore sem_c = 0; // binary semaphore

ProducerFunction() {
item produced;
while (true) {
        /* produce an item in "produced" */
        down(sem);
        Enqueue(produced);
        counter + + ;
        up(sem);
        up(sem_c);

    }
}

ConsumerFunction() {
item consumed;
while (true) {
        down(sem);
        if (counter == 0) {
            up(sem);
            down(sem_c);
            down(sem);
        }
        consumed = Dequeue( );
        counter - - ;
        up(sem);
        /* consume the item in "consumed" */
    }
}
```

כמות מוצרים

3

counter=1

counter X

הסבירו איך התוכנית פותרת(כולל כל התנאי פתרון קטע קריטי) את בעיית יצרנים-צרכנים או הפריכו ע"י דוגמא נגדית. האם בתוכנית יש שורות קוד מיותרות?

up(sem_c)

תשובה והסבר: אמנם יש מניעה הדדית [illegible] sem, כמו שצריך, אך יש בעיה אחרת.

התכנית [illegible] בעיית יצרנים-צרכנים [illegible]. לדוגמה, נניח שהמאגר ריק וצרכן מגיע, רואה שריק ומחכה [illegible] down(sem_c). כעת מגיע יצרן, מוסיף מוצר ומעיר את הצרכן, [illegible] context switch

לפני השורה down sem (בתוך ה-if). לאחר מכן מגיע יצרן נוסף, רואה counter=1, וצורך את המוצר. [illegible]

[illegible] sem, וצורך מוצר, [illegible] שהמאגר ריק, יש בעיה [illegible]

[illegible] counter, [illegible] up(sem_c) [illegible] כך שצרכן יחכה [illegible] כאשר המאגר אינו ריק.

[illegible] counter [illegible] down(sem_c) [illegible] if [illegible] הצרכן,

[illegible] שמחכה [illegible] המאגר ריק [illegible] את המניעה

[illegible] לבדוק אם המאגר ריק).

7

ד. (7 נקודות)

התוכנית שבסעיף ג׳ מופעלת באופן הבאה: אחרי זמן מה מכבים את כל היצרנים, אך הצרכנים ממשיכים לעבוד. מפעילים את היצרנים בחזרה רק מתי שכל הצרכנים נמצאים במצב המתנה.
האם ניתן לקבוע בוודאות מה הערך של counter במצב הזה?

תשובה והסבר:

במילה (מאחר ורק)
מצב אחד יכול להיות מקרה בו [illegible]
לא ניתן לקבוע בוודאות מה הערך של counter במצב הנ"ל, [illegible]

[illegible]

[illegible] שני הצרכנים רצים, עוצרים בגין
ה-if לפני השורה down(sem_c) (context switch), לאחר מכן מריצים יצרן אחד (נניח), counter=1 ו-sem_c=1. אז מפעילים יצרני
ומריצים שני הצרכנים, אחד מוריד את sem_c (נניח), counter=0, והשני נתקע על down(sem_c), כלומר counter=0
אך הוא לא [illegible] יכול אולי כיס, דוג' נוספת - אלה הצרכנים רצים בהמולה (מאחר ורק) ועוצרים (CNS) לפני
השורה down(sem_c), ואחרי זה מריצים [illegible] שני יצרנים, מעלים counter=2, sem_c=2 (פה הוא שמבוטל בינתיים, נניח), ואז
מכבים יצרנים, מריצים את שני הצרכנים, רק אחד יכול להוריד ולהכנס sem_c=0, ומעלים, counter=1, והשני
הצרכנים האחרים במצב המתנה על sem_c, אך counter=1, לכן לא ניתן לדעת בוודאות את ערכו
במצב זה.

המשך הבחינה בעמוד הבא

25
(7)

שאלה 8 (17 נקודות)

נניח שבמערכת ההפעלה ממומש אלגוריתם הדפדוף הבא:

כשמתרחשת חריגה(PAGE FAULT) בגישה לדף מספר NP ואין מסגרת פיזית פנויה, מפונה דף מספר N כזה ש MAX = | NP – N | (ערך מוחלט של ההפרש הגדול ביותר).

א. (7 נקודות)

כל כתובת - מס' דף   היסט offset

בשביל פשטות החישוב נניח שבמערכת קיימות 4 מסגרות פיזיות בגודל 1KByte ונתונה סדרת גישות לכתובות וירטואליות הבאות:

דף 1 דף 1 דף 0 דף 2 דף 3 דף 3 דף 4 דף 1 דף 0

1024,1040,512,2050,3100,3200,4500,1060,800

חשבו את מספר חריגות דף ומספר פינויי הדפים. בתשובה ציינו את מספר הדף שבו מתרחשת החריגה ואת מספר הדף המפונה.

(0-4) ✓

תשובה והסבר: חישוב מס' דף של כתובת וירטואלית x - $\lfloor \frac{x}{1024} \rfloor$. מעל כל כתובת ברשימה, כתבתי מס' דף.

1. הכתובת הוירטואלית 1024, דף 1 - לא קיים בזיכרון פיזי לכן חריגת דף מס' 1, והבאתו. (אין פינוי)
2. הכתובת 1040, דף 1, נמצא בזיכרון פיזי לכן אין חריגה ואין פינוי.
3. הכתובת 512, דף 0, לא נמצא לכן חריגת דף מס' 0, ומובא (אין פינוי).
4. הכתובת 2050, דף 2, לא נמצא לכן חריגת דף מס' 2, ומובא (אין פינוי).
5. הכתובת 3100, דף 3, לא נמצא לכן חריגת דף מס' 3, ומובא (אין צורך בפינוי).
6. הכתובת 3200, דף 3 נמצא לכן אין כלום.
7. הכתובת 4500, דף 4, לא נמצא לכן חריגת דף מס' 4, יש 4 מסגרות בזיכרון תפוסות לכן פינוי - מפונה דף 0 (מס' דף רחוק ביותר [illegible])
8. הכתובת 1060, דף 1, נמצא ולכן אין חריגה ואין פינוי.
9. הכתובת 800, דף 0, לא נמצא ולכן חריגת דף מס' 0, יש פינוי של דף מס' 4 (|4-0|=4 מקס').

סה"כ - 6 חריגות דף, 2 פינויי דפים. (4 ו-0)

ב.(5 נקודות)

לוקליות

איזה עיקרון התנהגות של תוכנית טיפוסית האלגוריתם הזה מנסה לנצל?

תשובה והסבר:

עיקרון ההתנהגות של תוכנית טיפוסית שהאלגוריתם מנסה לנצל הוא עיקרון הלוקליות, לפיו בכל רגע נתון תהליך משתמש במרחב זיכרון מצומצם בכתובות (לוקליות) קרובות זו לזו, כלומר אם תהליך משתמש בכתובת מסוימת סביר להניח שבזמן הקרוב ישתמש בכתובות קרובות, ולכן כשצריך לפנות, מפנים דף בעל מס' דף רחוק ביותר, סביר שלא נצטרך אותו בזמן הקרוב, ובכך נחסוך פינויים מיותרים ומספר חריגות דף ופינויי דפים, בגלל עיקרון...

ג. (5 נקודות)

השוו את האלגוריתם הזה עם LRU מבחינת ניצול עיקרון התנהגות תוכנית טיפוסית, ציינו את הדומה והשונה ביניהם.

תשובה והסבר

דומה - שניהם מנסים לחזות איזה דף מהזכרון כדאי להוציא כי הכי רחוק [illegible] [illegible]

חזרה לדף (כלומר דפים) עם שימוש בעקרון הלוקליות מסוים (כלומר או במקום אחר בזכרון)

אך שונה - אופן ניצול עקרון הלוקליות שונה בהתנהגות טיפוסית - LRU משתמש

בלוקליות בזמן (כל פעם שנשתמש בו בפעם הבאה לגבי זמן יחסית) כאשר האלגוריתם הזה בעקרון לוקליות

לוקליות במקום - בזכרון אחר (מה דף) (כלומר לוקליות בזכרון/מקום). בנוסף LRU מבוסס יותר

לשים לב זיכרון המחשב כך שכדאי יותר להסתמך על עקרון הלוקליות בתכנית טיפוסית. כיוון שזה

שאלגוריתם זה כל פעם לשים (רק שאיבר בה מה דף נמוך/גבוה ביותר יכול להספיק [illegible] חישוב [illegible]).

17
(8)

המשך הבחינה בעמוד הבא

שאלה 9 (13 נקודות)

התייחסו לשרת **בודד** להחזקת סרטים (דווקא סרטים, למשל YOUTUBE) עם 3 דיסקים מכאניים(מסתובבים עם ראש קריאה-כתיבה).

מבין שיטות הקצאת מקום לקבצים: רציפה, רשימה מקושרת, אינדקס(I-NODE) הציעו את השיטה היעילה ביותר להחזקת סרטים מבחינת:

ניצול מקום ←

א. ניצול מקום

ב. מהירות גישה.

התייחסו להבדלים באורך, פופולאריות ואופי הצפייה בסרטים. בין היתר ציינו מקום האכסון המועדף של הסרט.

אם צריך הניחו הנחות סבירות.

א. (7 נקודות)

יעילות ניצול מקום

תשובה והסבר:

ואז הגדלה עצמה

טבלת הקצאה-ו עשוי לבזבז הרבה מקום של מצביעים ✓, כאשר ובאם שסרטים עשויים להגיע לגדלים גדולים ולכן יידרשו כמה מצביעים לכל בלוק (indirect). ~~[illegible]~~ רשימה מקושרת תמנע מלהקצות מקום של מצביעים (כל בלוק), ולכן הגישה המועדפת עליה היא הקצאה רציפה, שכן אין בזבוז מקום של הרבה מצביעים, ואמנם בגישה זו יכול להיות ריסוק חיצוני, אך מכיוון שהסרטים כשהסרטים, הם בגודל לא ימחקו ויתווספו הרבה (סביר להניח) ולכן ניתן לקבוע מראש, כאשר ליצור ניתן כך לסדר ולחלק את ההקצאות [illegible] כי ארוכים והפופולאריים [illegible] סרטים, שכן האורך של סרט לא צפוי להשתנות בזמן (ולכן אין סכנת הרחבה), וניתן אז לחלק את הסרטים בין 3 הדיסקים על פי האורכים וכך למצער ריסוקים מראש, כאשר מכל [illegible] לבזבז זכרון של [illegible] ולבזבז (יותר) מאוד במקום). בנוסף אולי הגדיל כסרטים (הוא רצף וסדר) ולכן ניתן לשלול את הקרבה של מגזרים (וכן לדבר בזמן כשממקים קובץ [illegible] בהקצאה רשימה), ולהקצות [illegible] סדרתי ומקומי כך שסרטים פופולריים יהיו ברצף [illegible] וקרובים (אמנם מיקום [illegible] אך יש גם חישובי מבחן [illegible] מקום על אותו נסרים)

ב. (6 נקודות)

מהירות גישה

תשובה והסבר:

מבחינת מהירות גישה הגישה היעילה ביותר היא כאן בנוסף הקצאה רציפה, שכן כשמבוקש סרט כל הבלוק כולו בשאר [illegible] ברצף, ולא חוסך קריאות ולמנוע תנועות ראש מיותר (יקר), ובנוסף אורך סרט (לרב) (שאר קבוע), ולכן הקצאה רציפה יכולה להיות מגונן (חוץ מהסרטונים האלה והרחבות היא, גישה לו [illegible] קרוב גב), ובנוסף

~~[illegible]~~ [illegible]

את להביאם ולכוונן את הנבויה (למשל על סמך ניסיון עבר של ידין) ולמרכז הסרטים של מגו-ליינר קרבה יחסית לראש את הדיסק כך שנחסוך בזמני הגעת ראש מגנט. ובנוסף בגלל [illegible]

רשימה מקושרת-ו-מפתח עשויים לגרום תקלות, תנועות ראש מגנטי [illegible]

המשך בחינה בעמוד הבא

חסרה התייחסות לגודל הבלוק שמהווה שיקול

12

(9)

?? 10

חלק ג (15 נקודות)

ענו על השאלות 10-12.

(5 נקודות לשאלה; 15 נקודות לחלק זה).

שאלה 10

הניחו שקריאות המערכת SYSTEM CALLS מעבירות פרמטרים רבים לגרעין המערכת.

האם:

5
(10)

- לגרעין המערכת צריכה להיות גישה למרחב הזיכרון של תהליך המשתמש
- לתהליך המשתמש צריכה להיות גישה למרחב הזיכרון של גרעין המערכת

תשובה והסבר:

- כן, לגרעין המערכת צריכה להיות גישה למרחב הזיכרון של תהליך משתמש,
שכן הוא אחראי על ניהול המשאבים ~~הזיכרון~~ ההרגב של המכונה, וכן בקריאת SYSCALL
הגרעין צריך לקבל את הפרמטרים של הקריאה מהתהליך המשתמש (הוא צריך לשמור איפשהו לשימוש, וכו')
אחרכך אסור לתת לתהליך משתמש גישה לזיכרון (מרחב הגרעין), ~~[illegible]~~. כיוון שבכל קריאת מערכת וכן
הגרעין צריך לגשת ולשמור את מצב התהליך בו ~~[illegible]~~ (מידע) וכל מיני אלמנטים
- לא, אסור לתת לתהליך המשתמש גישה למרחב הזכרון של גרעין המערכת, שכן
כך יוכל ~~[illegible]~~ לגרום בעיות אבטחה, הגנה על משאבים ואמון הקבצים, וכל [illegible] לשמור בסודיות
זיכרונות וכו' - וכן כן גם תהליכים [illegible] ~~[illegible]~~ לכל [illegible] זכרון הגרעין,
אלא על ידי הפרוטוקולים הבטוחים והמוגדרים של מה שהגרעין להגן על המשאבים ויכיר... בדיוק כשכל ה-SYSCALL מעבירה את הפרמטרים הנדרשים לגרעין (והוא לוקח) את ... ומגבלות.

בקריאת (אחרים וכו').

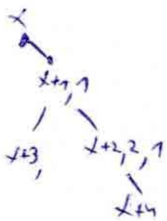
גישה (הגרעין) ולקריאת הגרעין.

שאלה 11

במערכת XV6 שתומכת בקונטיינרים קיימת קריאת מערכת kill(pid) המאפשרת הריגת תהליך ע"פ PID שלו. תזכורת: קיימות גם קריאות getpid() ו getppid().

נתון קוד הבא וניתן להניח שמרגע הרצתו במערכת לא נוצרו תהליכים מעבר לתהליך עצמו ומה שמופיע בקוד.

```
unshare(PID_NS);
pid = fork();  // Don't kill me
if (pid = 0)  {
   unshare(PID_NS);
   fork();
   fork();
   sleep(999999);
   }
else {
   ???
   }
```



נא לא לכתוב בשוליים

כתבו את הפקודות המדויקות שאמורות להיות איפה שמופיע ??? המאפשרות להרוג את **כל** התהליכים שנוצרו אחרי **Don't kill me** //

תשובה והסבר :

כיון ש- else נכנס תמיד [illegible], אין זכרון, ה-fork מחזיר ערך

שונה מ-0 (פיד הנוכחי לבן). ~~[illegible]~~ ב-pid יש את הפיד הנוכחי של

הבן הראשון אותו אין להרוג. הקריאה ל-fork הזו תייצר ~~[illegible]~~ נכד (קונטיינר חדש, אך

לא חיצוני) ~~[illegible]~~ גם הנכד וגם הבן מריצים עוד fork כך יש עוד נכד 2, ועוד בן

הנכדה. ~~[illegible]~~ kill [illegible] את הפיד [illegible] בתוך הקונטיינר, ולכן זה המבוקש של ה-pid

הנוכחי (הנכדים והבן) של הפידים שיש להרוג:

kill(pid+1)

kill(pid+2)

kill(pid+3)

5
(11)

(שכן הקצאת הפידים נעשה
סדרתי ויחידי

שאלה 12

הביאו 3 דוגמאות לקריאות מערכת שצריך לשנות את אופן מימוש כשמוסיפים למערכת הפעלה תמיכה בקונטיינרים. תארו בקצרה את השינוי הנדרש.

תשובה והסבר :

~~[illegible]~~ write(), read() - יש לוודא לכך שאין גישה לאותו mountFS וכיוצא

בזה.

כך getpid() מה שמוחזר צריך להתאים לנתוני pidns ו-pid תקין.

5
(12)

בהצלחה!

while
vcpu
[illegible]
PS
סיגנל
[illegible]

שאלון 514

87.72.9 M2